

MULTIMODAL DOCUMENT INTELLIGENCE SYSTEM FOR INVOICE AND RECEIPT PROCESSING USING VISION-LANGUAGE MODELS



A PROJECT REPORT

Submitted by

AADHISESHAN S	(732722104001)
JAYASIMBU J	(732722104018)
REMO V	(732722104047)
SRIGANTH GV	(732722104056)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

SRI SHANMUGHA COLLEGE OF ENGINEERING AND TECHNOLOGY

(AN AUTONOMOUS INSTITUTION)

SANKARI - 637 304.

ANNA UNIVERSITY: CHENNAI 600 025

MAY 2026

BONAFIDE CERTIFICATE

Certified that this project report “**MULTIMODEL DOCUMENT INTELLIGENCE SYSTEM FOR INVOICE AND RECEIPT PROCESSING USING VISION-LANGUAGE MODEL WITH LAYOUT AWARE-OCR**” is the Bonafide work of **AADHISESHAN S(732722104001), JAYASIMBU J(732722104018), REMO V (732722104047), SRIGANTH GV (732722104056)** who carried out the project work under my supervision.

SIGNATURE

Dr. P. NANDHINI, M.E., Ph.D.,

**HEAD OF THE DEPARTMENT,
ASSOCIATE PROFESSOR,
Department of Computer Science and
Engineering,
Sri Shanmugha College of Engineering
and Technology,
Sankari - 637 304**

SIGNATURE

Dr. P. NANDHINI, M.E., Ph.D.,

**SUPERVISOR,
ASSOCIATE PROFESSOR,
Department of Computer Science and
Engineering,
Sri Shanmugha College of Engineering and
Technology,
Sankari - 637 304**

Submitted for the End Semester Project Viva-Voce Examination held on

Internal Examiner

External Examiner

DECLARATION

We affirm that the project work titled “**MULTIMODEL DOCUMENT INTELLIGENCE SYSTEM FOR INVOICE AND RECEIPT PROCESSING USING VISION-LANGUAGE MODEL WITH LAYOUT AWARE-OCR**” being submitted in partial fulfillment for the award of Bachelor of Engineering degree is the original work carried out by us. It has not formed the part of any other project work submitted forward of any degree or diploma, either in this or any university.

AADHISESHAN S (732722104001)

JAYASIMBU J (732722104018)

REMO V (732722104047)

SRIGANTH GV (732722104056)

(SIGNATURE OF THE CANDIDATES)

1)

2)

3)

4)

I certify that the declaration made above by the candidates is true.

(Signature of the Supervisor)

Dr.P. NANDHINI,M.E.,Ph.D.,

ACKNOWLEDGEMENT

First and foremost, our thanks to the lord almighty, the great architect of the universe, who has blessed us to complete this project.

We wish our heartfelt thanks to our respected founder and Chairman of Sri Shanmugha College of Engineering and Technology **Thiru K.SHANMUGAM** and the Vice-Chairman of Sri Shanmugha College of Engineering and Technology **Mr.S.PRABAHAR,B.E.**, for giving as an opportunity and facilities for the completion of this project at this institution.

We wish to express our deep sense of gratitude to Executive Director **Mr.THIRUMOORTHY ARUMUGAM,M.E.**, for providing an opportunity to work on this project.

We are expressing our extreme gratitude and heartfelt thanks to the respect Principal **Dr.G NALLAKUMARASAMY,M.TECH.,Ph.D.**, of the college facilities to do this project effectively for allowing us to have extensive use.

We would like to express our profound interest and sincere gratitude to our respected Head of the Department **Dr.P.NANDHINI,M.E.,Ph.D.**, for her valuable guidance and constant support in the successful completion of the project.

We feel proud and honored in extending our sincere thanks to our project supervisor **Dr.P.NANDHINI,M.E.,Ph.D.**, for being more informative and providing guidelines regarding this project all the time. Finally, we thank our friends and family members for their kind completion of our project.

ABSTRACT

Real-world Indian financial documents such as invoices and receipts present significant challenges due to irregular layouts, multilingual text, faded print, and overlapping elements. Traditional Optical Character Recognition (OCR) systems struggle to accurately extract structured information from such noisy and unstructured data. To address this limitation, this paper proposes a Multimodal Document Intelligence System that leverages the Qwen2.5-VL-32B Vision-Language Model for direct image-to-JSON extraction without relying on conventional OCR pipelines. The system integrates advanced preprocessing techniques such as Contrast Limited Adaptive Histogram Equalization (CLAHE) and introduces a Digital Twin representation for layout-preserving verification. To overcome hardware constraints, a Master-Worker architecture is designed, where lightweight local processing is combined with cloud-based GPU inference via secure tunnelling. Experimental results demonstrate significant improvements in extraction accuracy, achieving over 92% key-value accuracy on complex documents, outperforming traditional OCR-based approaches. The platform supports multilingual invoice understanding across all Indian languages and provides accurate field extraction for vendor details, invoice numbers, dates, tax values, item descriptions, and total amounts. The final output is presented as structured JSON data and Digital Twin layout representations through an interactive React-based dashboard. The proposed system significantly improves extraction accuracy, reduces manual data entry effort, supports multilingual document processing, and enables scalable AI-powered document intelligence for modern business applications.

TABLE OF CONTENTS

CHAPTER NO	TITLE	PAGE NO
	ABSTRACT	v
	LIST OF FIGURES	viii
	LIST OF ABBREVIATIONS	ix
1.	INTRODUCTION	1
	1.1 OBJECTIVE	2
	1.2 AIM	2
2.	LITERATURE SURVEY	3
3.	SYSTEM STUDY	6
	3.1 EXISTING SYSTEM	6
	3.2 DISADVANTAGES OF EXISTING SYSTEM	7
	3.3 PROBLEM STATEMENT	7
	3.4 PROPOSED SYSTEM	8
	3.5 ADVANTAGES OF PROPOSED SYSTEM	9
4.	METHODOLOGY	10
	4.1 EXISTING METHOD	10
	4.2 PROPOSED METHOD	12
	4.2.1 KEY FEATURES OF THE PROPOSED METHOD	12
5.	SYSTEM DESIGN	14
	5.1 OVERALL SYSTEM ARCHITECTURE	14
	5.2 DATA FLOW DESIGN	14
	5.3 MODEL ARCHITECTURE DESIGN	15
	5.4 DEPLOYMENT DESIGN	15

5.5	BLOCK DIAGRAM	16
5.6	ACTIVITY DIAGRAM FOR USER	17
5.7	SEQUENCE DIAGRAM	18
5.8	DATAFLOW DIAGRAM	19
5.9	USECASE DIAGRAM	20
5.10	ACTIVITY DIAGRAM	21
6.	IMPLEMENTATION	22
6.1	TOOLS	22
6.2	IMPLEMENTATION	24
6.2.1	USER INTERFACE IMPLEMENTATION	24
6.2.2	DOCUMENT UPLOAD MODULE	24
6.2.3	IMAGE PREPROCESSING MODULE	24
6.2.4	OCR TEXT EXTRACTION MODULE	24
6.2.5	LAYOUT ANALYSIS MODULE	24
6.2.6	FIELD EXTRACTION MODULE	25
6.2.7	STRUCTURED OUTPUT GENERATION	25
6.2.8	DATABASE STORAGE MODULE	25
6.2.9	RESULT DISPLAY AND DOWNLOAD	25
7.	SCOPE OF FUTURE ENHANCEMENT	26
8.	CONCLUSION	28
	REFERENCE	29
	APPENDIX	30
	A. CODE	30
	B. SCREENSHOTS	36

LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO
5.1	Block Diagram	16
5.2	Activity Diagram For User	17
5.3	Sequence Diagram	18
5.4	Dataflow Diagram	19
5.5	Use Case Diagram	20
B.1	Home Screen	36
B.2	Invoice Upload Screen	37
B.3	Processing Screen	38
B.4	Output Screen	39

LIST OF ABBREVIATIONS

ABBREVIATIONS	DEFINITION
OpenCV	Open-Source Computer Vision Library
JSON	JavaScript Object Notation
VLM	Vision-Language Model
LayoutLMv3	Layout Language Model Version3
OCR	Optical Character Recognition
ATS	Applicant Tracking System
MCP	Model Context Protocol
API	Application Programming Interface
NLP	Natural Language Processing

CHAPTER 1

INTRODUCTION

In modern enterprises, automated document processing plays a critical role in accounting, auditing, and supply chain operations. However, real-world documents, especially in the Indian context, are highly unstructured and often contain multilingual text, inconsistent formatting, and degraded visual quality

Traditional Optical Character Recognition (OCR) systems attempt to solve this problem using a two-stage pipeline: first, detecting text regions through bounding box localization, and second, recognizing the textual content within those regions. While this approach works reasonably well for clean, well-formatted documents, it struggles in real-world conditions. Errors in text detections such as missing or incorrectly segmented regions—propagate to the recognition stage, resulting in incorrect or incomplete outputs. Furthermore, OCR systems lack an inherent understanding of document structure and context. They often fail to preserve relationships between key elements, such as associating item names with their corresponding prices in a receipt, leading to loss of semantic meaning.

To overcome these challenges, this work proposes a Vision-Language Model (VLM)-based system that fundamentally redefines the document processing pipeline. Instead of relying on intermediate text extraction steps, the proposed approach directly interprets document images at the pixel level and generates structured outputs in formats such as JSON. By leveraging advanced multimodal learning, the system simultaneously captures both textual content and spatial relationships within the document. This enables a deeper contextual understanding, allowing the model to accurately identify key-value pairs, tabular data, and hierarchical structures without explicit layout rules.

1.1 OBJECTIVE

The main objective of this research is to develop an efficient and intelligent **Multimodal Document Intelligence System** capable of accurately extracting structured information from complex, real-world financial documents such as invoices and receipts. The system aims to overcome the limitations of traditional OCR-based approaches by leveraging advanced Vision-Language Models (VLMs) for direct image understanding and structured data generation

1.2 AIM

The aim of this project is to design and implement an advanced Multimodal Document Intelligence System that can efficiently process and understand complex financial documents such as invoices and receipts. The system aims to utilize Vision-Language Models (VLMs) to directly interpret document images and generate structured outputs without relying on traditional OCR pipelines .It focuses on achieving high accuracy, layout awareness, and multilingual understanding, enabling reliable extraction of key information from real-world, unstructured documents. The system is intended to reduce manual data entry, improve processing speed, and provide a scalable, cost-effective solution for enterprise document automation.

CHAPTER 2

LITERATURE SURVEY

1. Vision Transformer for Image Understanding – Dosovitskiy et al. (2021).

This study introduced Vision Transformer (ViT), which processes images as patch sequences instead of using traditional convolution methods. It helps models understand visual patterns and spatial relationships in images. This concept supports our system because invoices and receipts need layout-aware visual understanding. [1].

2. Attention Mechanism – Vaswani et al. (2017).

This paper proposed the Transformer architecture using self-attention. Self-attention helps models focus on important parts of input data. In our project, attention-based visual understanding helps connect related fields such as item names, prices, subtotal, tax, and total amount. [2].

3. Qwen-VL: Vision-Language Model – Qwen Team (2023).

Qwen-VL is a vision-language model capable of understanding images and text together. It can read document images, recognize layout, and generate structured responses. This is highly relevant to our project because we use VLM-based extraction instead of depending only on OCR [3].

4. LayoutLM for Document Understanding – Xu et al. (2020).

This LayoutLM combines text and layout information for document image understanding. It improves extraction from forms and invoices by considering word positions. However, it still depends on OCR output, so OCR errors can affect final results. [4].

5. Donut: OCR-Free Document Understanding – Kim et al. (2022).

Donut introduced an OCR-free approach for document understanding by directly converting document images into structured text. It reduces dependency on OCR engines. But it may struggle with high-resolution variations and complex layouts, which our VLM-based system aims to handle better [5].

6. Adaptive Histogram Equalization – Pizer et al. (1987).

This study introduced adaptive histogram equalization techniques to improve image contrast. In our project, CLAHE is used to enhance faded receipts and improve visibility before sending the image to the model. This helps improve extraction accuracy from low-quality documents [6].

7. LLaVA: Visual Instruction Tuning – Liu et al. (2023).

LLaVA demonstrated how visual instruction tuning improves image-based question answering and reasoning. It shows the strength of combining vision models with language models. This supports our project's idea of using prompts to extract structured invoice and receipt data. [7].

8. Tesseract OCR Engine (2022).

Tesseract is one of the most widely used OCR engines for text extraction from images. It performs well on clean scanned documents but struggles with faded thermal receipts, multilingual text, overlapping stamps, and irregular layouts. These limitations motivate our VLM-based approach [8].

9. LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking – Huang et al. (2022).

This paper presents LayoutLMv3 improves document AI by combining text and image masking. It performs well in form understanding, receipt understanding, and document layout analysis. [9].

10. Visual Instruction Tuning / LLaVA – Liu et al. (2023).

This research explores the integration of AI models in wearable devices for continuous ECG monitoring. It highlights the need for fast, lightweight, and accurate models for real-time health monitoring applications [10].

11. PaddleOCR 3.0 Technical Report – Cui et al. (2025).

PaddleOCR 3.0 focuses on multilingual OCR, document parsing, and key information extraction. It is useful for invoice and receipt processing, but still follows an OCR-based pipeline [11].

12. PaddleOCR-VL: Multilingual Document Parsing – Cui et al. (2025)

PaddleOCR-VL combines a visual encoder and language model for document parsing. It is designed for multilingual document understanding and structured extraction. [12].

13. Multimodal Document Intelligence System using Qwen2.5-VL – Proposed Work (2026).

This work uses Qwen2.5-VL-32B to directly interpret invoice and receipt images and generate structured JSON output. It avoids traditional OCR dependency and supports Indian multilingual documents. [13].

14. Master-Worker Architecture for VLM-based Document Extraction – Proposed Work (2026).

The proposed system uses a local FastAPI master node and a remote Kaggle GPU worker node through Cloudflare Tunnel. This enables high-end VLM processing without expensive local hardware. [14].

15. An Image is Worth 16x16 Words: Vision Transformer – Dosovitskiy et al. (2021)

This paper introduced Vision Transformer, where images are processed as patches. It supports the idea of understanding document images visually. [15].

CHAPTER 3

SYSTEM STUDY

3.1 EXISTING SYSTEM

- **OCR-Based Systems (Tesseract, EasyOCR)**

In OCR (Optical Character Recognition) systems are widely used for extracting text from document images. These systems follow a two-step process involving text detection and recognition. They work well for clean and structured documents but perform poorly when dealing with noisy images, faded receipts, and complex layouts. Additionally, OCR systems extract only raw text and fail to preserve the spatial relationships between different elements in the document the current system.

- **Rule-Based Parsing Methods**

Rule-based systems extract specific fields using predefined patterns and keywords such as “Total”, “Date”, or “Invoice Number”. While they are simple to implement, they require manual configuration for each document format. These systems are not scalable and fail when document structure changes, making them unsuitable for handling diverse real-world documents.

- **Inefficient Deployment on Edge Devices**

Existing mode systems do not consider the spatial arrangement of text within a document. They fail to recognize the relationship between different sections such as headers, tables, and line items. Since invoices and receipts often follow semi-structured formats, the absence of layout awareness leads to incorrect mapping of values and reduces overall extraction accuracy.

- **Manual Data Entry Requirement**

Due to the limitations of OCR systems, extracted data often requires manual verification and correction. Users must manually identify and input key details such as invoice number, date, and total amount. This increases processing time, introduces human errors, and reduces efficiency in large-scale document handling environments

- **Poor Performance on Handwritten and Noisy Documents**

Traditional OCR systems perform poorly when handling handwritten receipts, low-quality scans, or images with noise, shadows, or distortions. Variations in handwriting styles and image quality significantly affect text recognition accuracy, making the system unreliable for practical use.

3.2 DISADVANTAGES OF EXISTING SYSTEM

- Entire system output depends on OCR quality; small errors lead to incorrect final extraction.
- Fails to handle faded receipts, blurred images, and low-quality scans effectively.
- Cannot preserve spatial relationships like item–price alignment in receipts.
- Struggles with irregular formats, overlapping text, and borderless tables.
- Performs poorly on documents containing mixed languages like English + Tamil/Hindi.
- Requires manual rules for each format, making it difficult to adapt to new document types.
- Extracts only text without understanding meaning or relationships between fields.

3.3 PROBLEM STATEMENT

At present, In real-world applications, invoices and receipts are not in a fixed format and often contain noise, faded text, and multiple languages. Traditional OCR-based systems try to extract text, but they fail when the document quality is poor or the layout is complex. These systems also cannot maintain the correct relationship between data like item and price. Rule-based methods are not flexible and fail when the format changes. Even advanced models depend on OCR, so errors still occur. Hence, a better system is needed to accurately understand documents and extract structured data directly.

3.4 PROPOSED SYSTEM

The proposed system uses a **Vision-Language Model (VLM)** to directly understand document images and extract structured data without using traditional OCR. It can handle complex layouts, faded text, and multilingual content more effectively. The system captures both text and spatial relationships, ensuring accurate mapping of information like item and price.

It also generates structured output in JSON format along with a Digital Twin for easy verification. A Master-Worker architecture is used to perform heavy processing in the cloud while keeping the local system lightweight. This approach improves accuracy, reduces errors, and supports real-time document processing. The system includes the following key features:

- **OCR-Free Document Understanding**

The system directly processes document images using Vision-Language Models. This avoids OCR errors and improves overall extraction accuracy.

- **Layout-Aware Extraction**

The model understands both text and its position in the document. This helps in correctly mapping relationships like item and price.

- **Multilingual Support**

The system can handle documents with multiple languages. It accurately extracts and standardizes mixed-language content.

- **Digital Twin Representation**

Generates a layout-preserving text grid of the document. This makes manual verification simple and user-friendly.

3.5 ADVANTAGES OF PROPOSED SYSTEM

- Direct image understanding reduces errors compared to OCR-based systems.
- Works well with irregular layouts, tables, and overlapping text.
- Supports mixed-language documents like English + Tamil/Hindi.
- Maintains spatial relationships such as item and price alignment.
- Automatically extracts structured data, minimizing human intervention.
- Can adapt to different document formats without rule changes.
- Uses GPU-based processing for faster and powerful computation.
- Enables quick extraction suitable for enterprise applications.

CHAPTER 4

METHODOLOGY

4.1 EXISTING METHOD

The existing document processing systems are mainly based on traditional OCR and rule-based extraction approaches. These systems attempt to extract text from invoices and receipts using OCR engines and then apply predefined rules for identifying important fields. However, such systems are not efficient in handling complex document layouts, multilingual content, noisy images, and real-world unstructured financial documents. The overall workflow of the existing methods is explained below.

- **Data Acquisition**

In existing systems, invoice or receipt images are collected from scanners, mobile devices, or uploaded PDF files. These documents may contain noise, blur, faded text, handwritten content, or inconsistent layouts, which affect extraction quality.

- **OCR-Based Text Extraction**

Traditional systems use OCR engines such as Tesseract or PaddleOCR to extract raw text from the document image. While OCR helps in converting image content into text, the extracted output often contains recognition errors, especially in low-quality or multilingual documents..

- **Preprocessing**

Basic preprocessing techniques such as resizing, filtering, and noise removal are applied to improve image quality before OCR processing. However, many existing systems do not apply advanced enhancement techniques required for handling real-world invoice and receipt images effectively.

- **Rule-Based Parsing**

After OCR extraction, predefined rules and templates are used to identify fields like invoice number, vendor name, date, and total amount. These methods work only for fixed document formats and fail when the layout changes or when documents contain irregular structures.

- **Layout Processing**

Some advanced systems use layout-aware models such as LayoutLM or LayoutLMv3 to understand document structure using OCR text and bounding box information. Although these models improve layout understanding, they still depend heavily on OCR quality, which leads to error propagation.

- **Data Extraction**

The processed text is analyzed to extract important details such as item names, prices, taxes, and totals. However, existing systems often struggle to maintain the correct relationships between fields, especially in tables and complex invoice layouts.

- **Limitations of Existing Systems**

Existing systems have high dependency on OCR accuracy and are not suitable for highly unstructured or multilingual documents. They require manual configuration, struggle with noisy images, and often fail to preserve semantic and spatial relationships within documents.

4.2 PROPOSED METHOD

To overcome the limitations of traditional OCR-based document processing systems, an efficient **Multimodal Document Intelligence System** is proposed. The proposed system combines OCR, layout-aware document understanding, and Vision-Language processing techniques to accurately extract structured information from invoices and receipts. This approach improves document understanding, preserves layout relationships, and supports multilingual financial documents.

The system uses advanced technologies such as **Tesseract/PaddleOCR** for text extraction, **LayoutLMv3** for layout-aware document understanding, and **Hugging Face Vision-Language models** for contextual information extraction. The backend is developed using **Python and FastAPI**, while **React** is used for building the frontend interface.

Extracted information is converted into structured **JSON format** and stored in **MongoDB** for efficient retrieval and management. This approach improves extraction accuracy, supports multilingual documents, and handles complex layouts more effectively than traditional systems.

4.2.1 Key Features of the Proposed Method

- **OCR-Based Text Extraction**

The system uses Tesseract and PaddleOCR to extract text from invoice and receipt images accurately. It supports printed and multilingual documents with improved text recognition capability.

- **Layout-Aware Document Understanding**

LayoutLMv3 analyzes both textual content and spatial layout information together. This helps the system understand tables, headers, line items, and field relationships correctly.

- **Multilingual Document Support**

The proposed system can process documents containing multiple languages and mixed text formats. It improves extraction accuracy for real-world invoices and receipts used in different regions

- **Structured JSON Output Generation**

Extracted document information is converted into structured JSON format automatically. This makes the output easy to store, analyze, and integrate with enterprise applications.

- **Image Enhancement and Noise Removal**

Preprocessing techniques such as resizing and noise reduction improve image clarity before OCR. This helps the system handle blurred, faded, and low-quality receipt images effectively.

- **Real-Time Result Display**

The extracted information is displayed instantly through the React frontend interface. Users can easily verify, review, and export the processed document data.

- **Scalable Backend Processing**

FastAPI-based backend enables fast API communication and efficient document processing. The system architecture supports scalable and real-time enterprise-level deployment.

CHAPTER 5

SYSTEM DESIGN

5.1 OVERALL SYSTEM ARCHITECTURE

The overall system architecture of the proposed Multimodal Document Intelligence System is designed to perform intelligent invoice and receipt processing efficiently. The architecture consists of frontend, backend, OCR engine, layout-aware processing model, Vision-Language understanding module, and database storage components. The React frontend allows users to upload invoice or receipt documents, while the FastAPI backend handles request processing and communication between modules.

OCR engines such as Tesseract and PaddleOCR extract textual information from the uploaded document image. LayoutLMv3 and Hugging Face Vision-Language models analyze the document structure, layout relationships, and contextual information to generate structured outputs. Finally, the extracted data is converted into JSON format and stored securely in MongoDB for future access and result management.

5.2 DATA FLOW DESIGN

The data flow design describes how information moves through different stages of the proposed system. Initially, the user uploads invoice or receipt images through the frontend interface. The uploaded document is sent to the backend, where preprocessing techniques such as resizing, contrast enhancement, and noise removal are applied.

Next, LayoutLMv3 analyzes layout structure and spatial relationships between fields. The Vision-Language processing module further understands document context and extracts important details such as vendor name, invoice number, date, item details, tax, and total amount. The extracted information is validated, converted into structured JSON format, stored in the database, and finally displayed to the user through the frontend interface.

5.3 MODEL ARCHITECTURE DESIGN

The model architecture of the proposed system combines OCR, layout-aware document understanding, and Vision-Language processing techniques. Tesseract and PaddleOCR are used as the OCR layer for extracting text from document images.

The LayoutLMv3 acts as the layout-aware transformer model that processes both textual and spatial information together to understand document structure effectively. Hugging Face Vision-Language models further analyze visual and textual features to improve contextual understanding and extraction accuracy.

The extracted data passes through post-processing and validation modules before generating the final structured output. This hybrid architecture improves performance in handling noisy, multilingual, and complex invoice documents.

5.4 DEPLOYMENT DESIGN

The deployment design of the proposed system is based on a client-server architecture. The frontend interface is developed using React and deployed as a web application for user interaction. The FastAPI backend is deployed as the server-side processing layer that manages API communication and document processing workflows.

OCR engines, LayoutLMv3, and Vision-Language models are integrated within the backend environment using Python and Hugging Face libraries. MongoDB is used as the database for storing extracted JSON data and processed document results. The deployment architecture supports scalable, secure, and real-time invoice processing suitable for enterprise-level applications.

5.5 BLOCK DIAGRAM

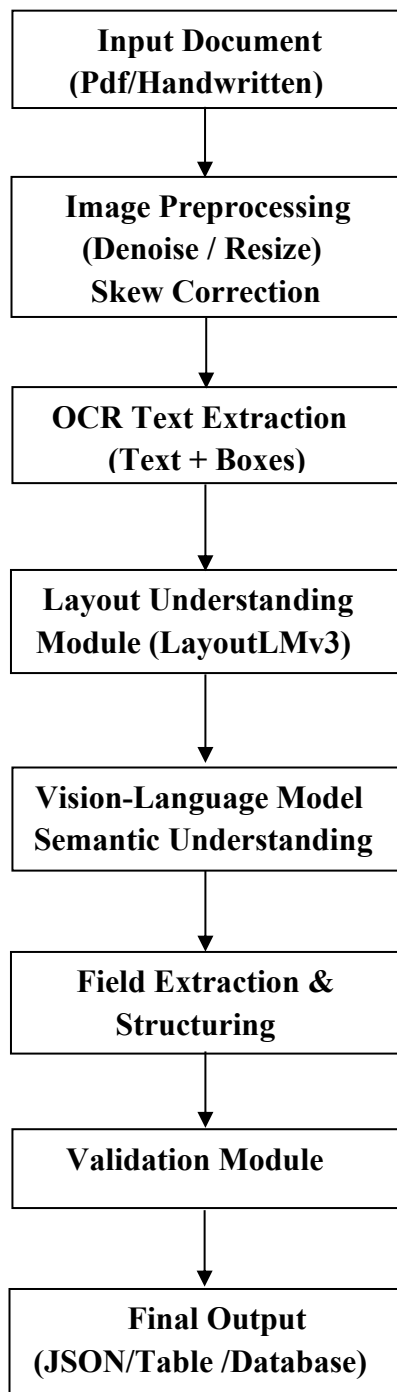


Fig.No:5.5 Block diagram

This block diagram shows the overall system flow from document upload to structured output generation. The uploaded invoice/receipt is processed using OCR, layout-aware understanding, and VLM-based extraction to produce accurate JSON results.

5.6 ACTIVITY DIAGRAM FOR USER

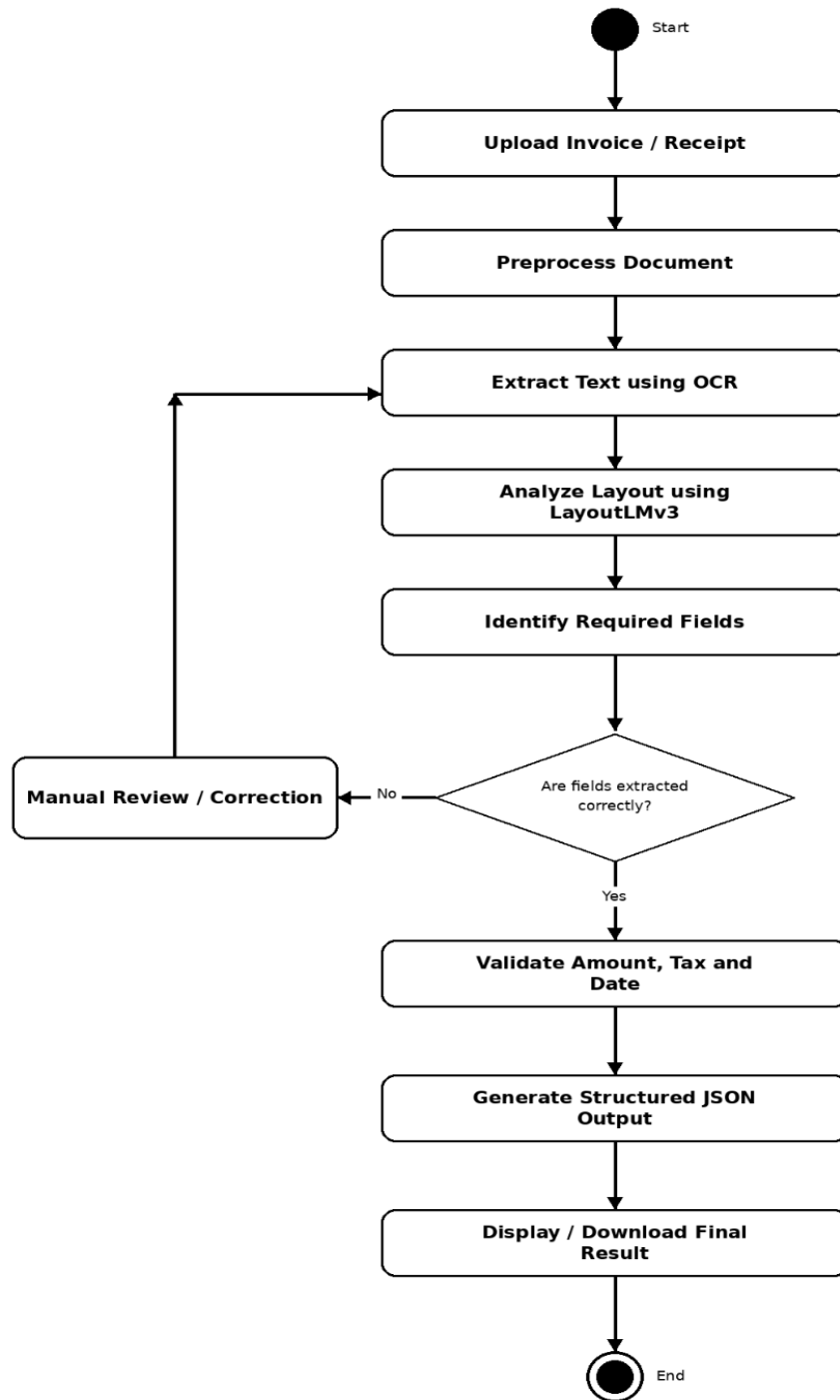


Fig.No:5.6 Activity diagram for user

The activity diagram explains the step-by-step working process of the system. It starts with user document upload, then moves through OCR extraction, layout analysis, field extraction, validation, and finally displays the processed output.

5.7 SEQUENCE DIAGRAM

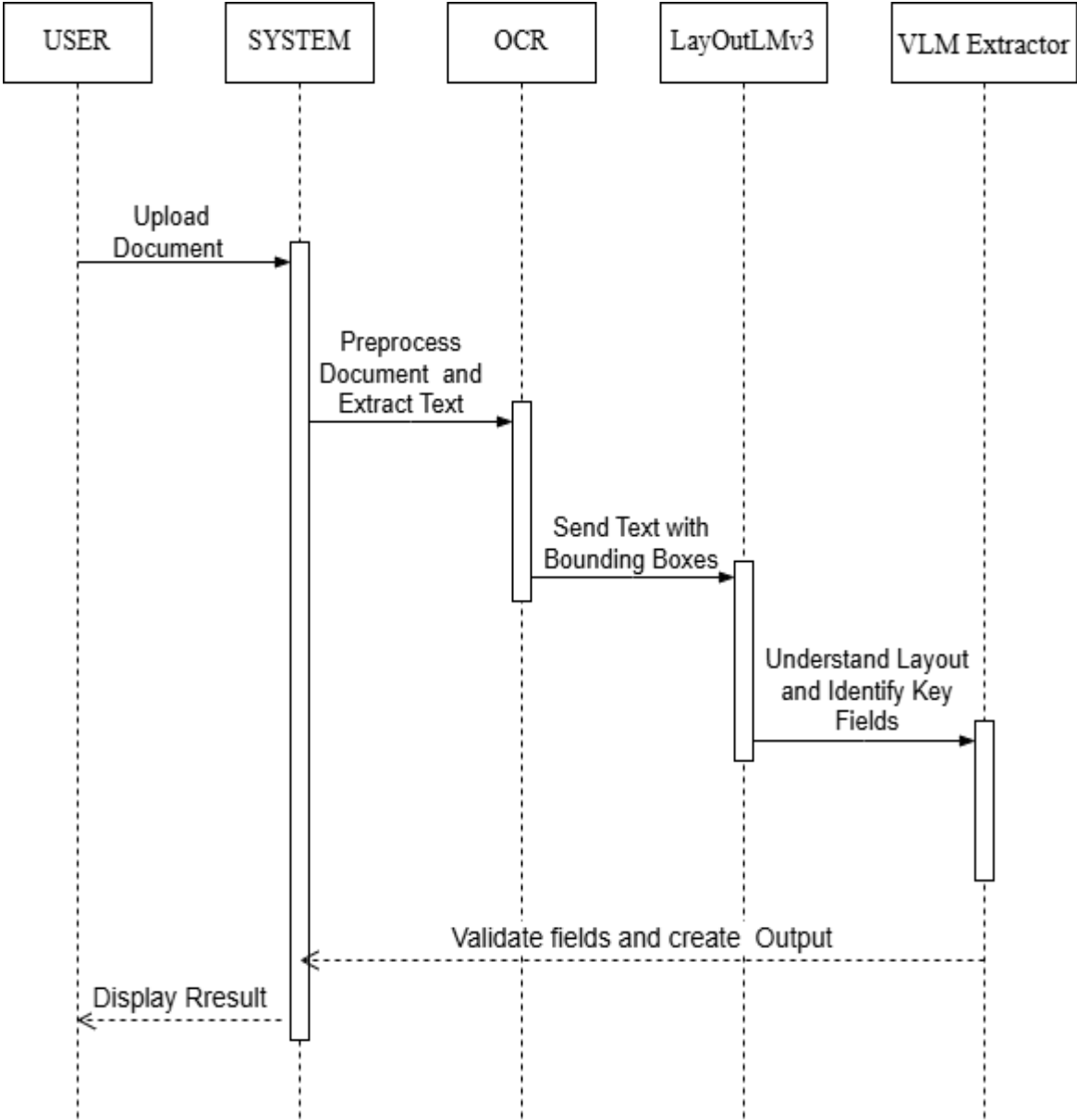


Fig.No:5.7 Sequence diagram

The sequence diagram shows the interaction between the user, frontend, backend, OCR engine, AI model, and database. It clearly explains how each component communicates during invoice and receipt processing.

5.8 DATAFLOW DIAGRAM

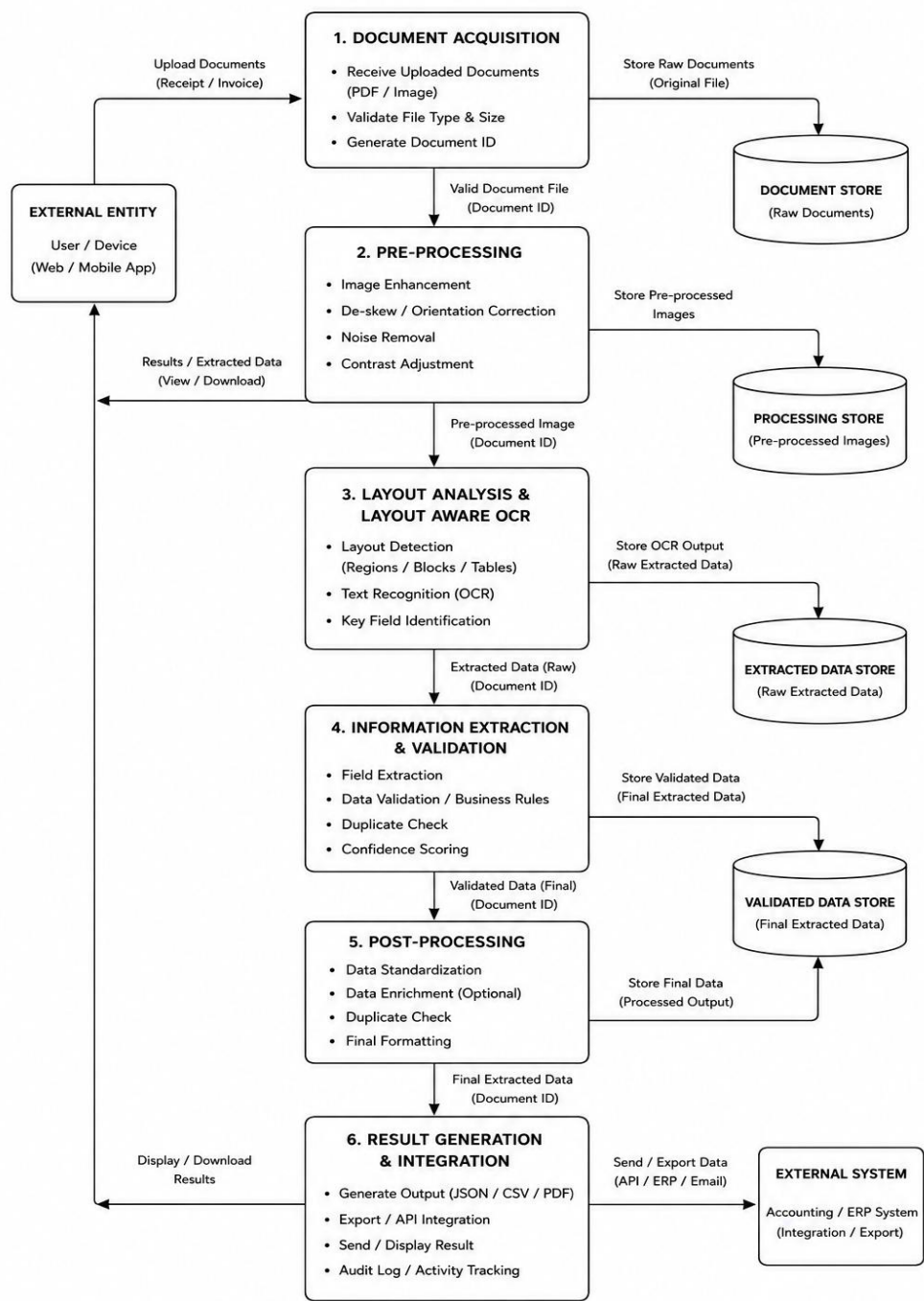


Fig.No:5.8 Dataflow diagram

The data flow diagram represents how data moves inside the system. It shows how input documents are converted into extracted text, analyzed layout information, structured fields, and final output.

5.9 USE CASE DIAGRAM

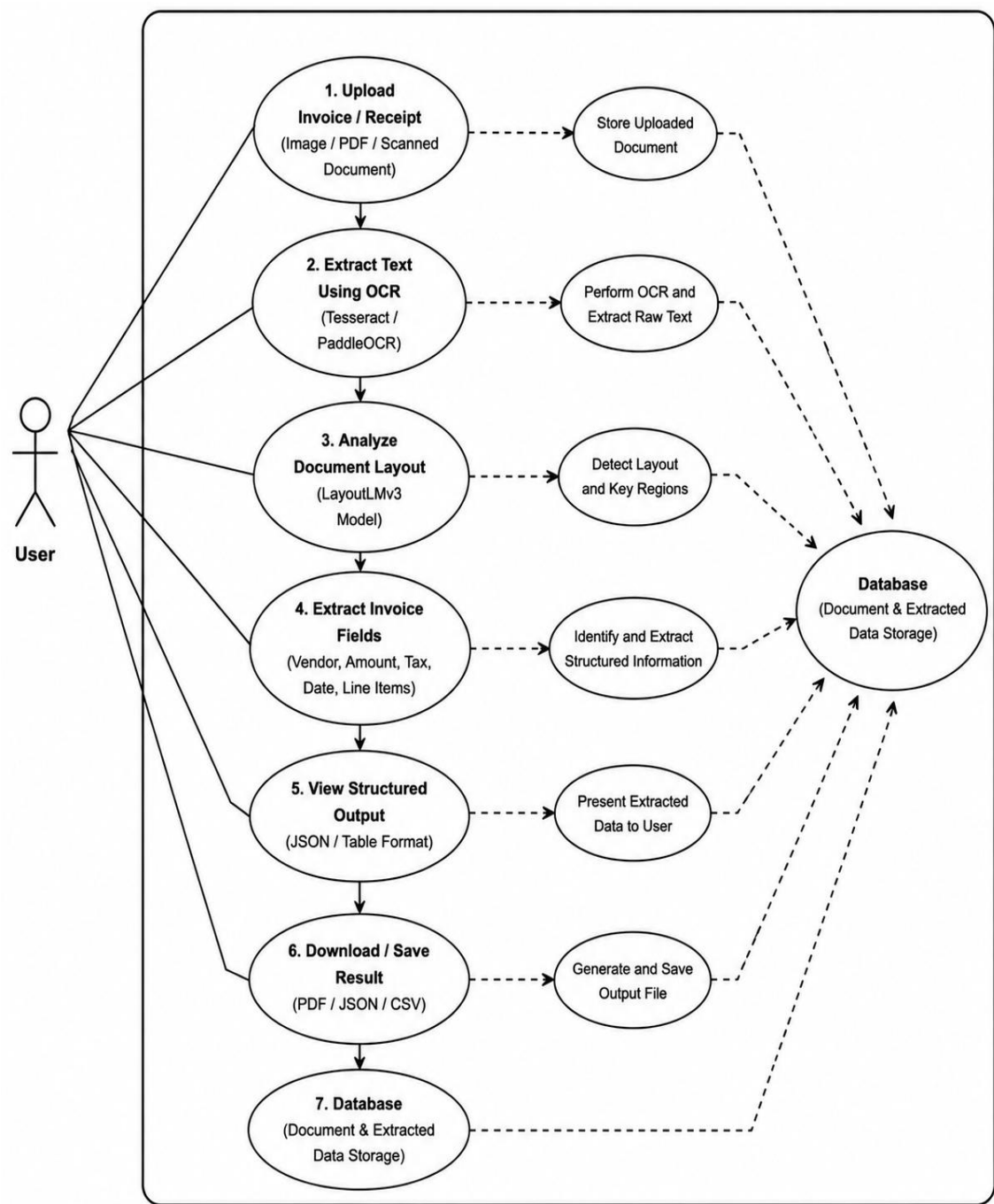


Fig.No:5.9 Usecase diagram

The use case diagram shows the main actions performed by the user and the system. The user uploads documents and views results, while the system performs OCR, layout understanding, field extraction, and output generation

5.10 ARCHITECTURE DIAGRAM

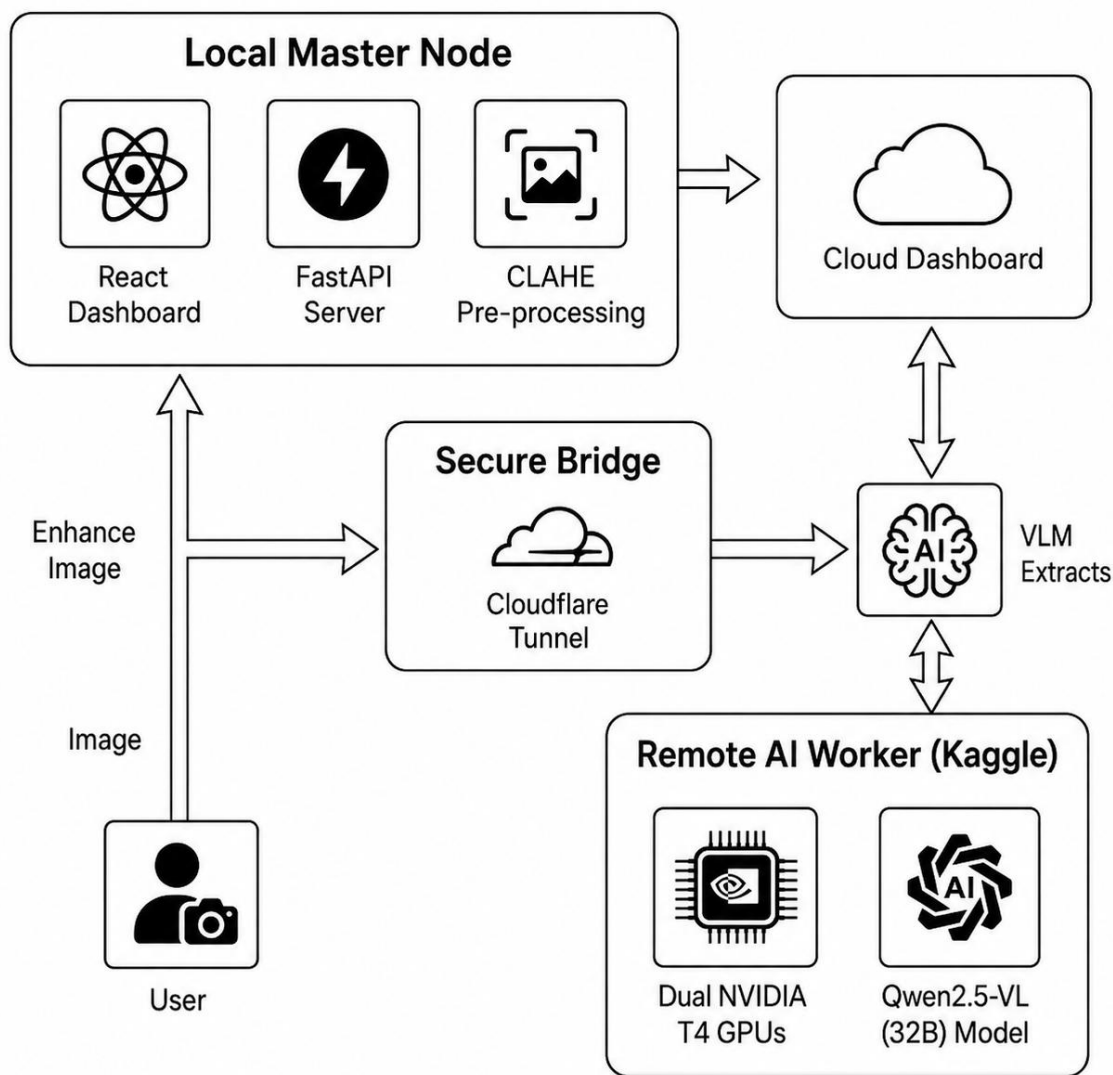


Fig.No:5.10 Architecture Diagram

This diagram shows the workflow of the proposed AI-based image processing system. The user image is enhanced in the local server, securely transferred through Cloudflare Tunnel, and processed by the Qwen2.5-VL model in the remote AI worker to extract results and display them on the cloud dashboard.

CHAPTER 6

IMPLEMENTATION

6.1 TOOLS

- **Backend – Python**

Python is used as the core backend language to implement the complete processing pipeline. It handles OCR integration, model execution, and data processing efficiently. Its rich ecosystem of libraries makes it suitable for AI-based applications.

- **Layout-Aware Model – LayoutLMv3**

LayoutLMv3 is used for understanding the document structure and layout information from invoices and receipts. It helps identify headers, tables, key-value pairs, and line-item regions accurately using both text and positional information.

- **OCR Engine – Tesseract OCR / PaddleOCR**

Tesseract OCR and PaddleOCR are used to extract textual content from scanned documents, images, and PDFs. These OCR engines convert visual text into machine-readable format with support for multilingual document processing.

- **Vision-Language Model (VLM)**

The Vision-Language Model is used to analyze both visual and textual information together. It intelligently extracts important invoice fields such as vendor name, invoice number, amount, tax details, and dates from different document formats.

- **Model Integration – Hugging Face Transformers**

Hugging Face Transformers is used for integrating pretrained AI models into the system. It provides easy access to LayoutLMv3 and other transformer-based document understanding models for efficient AI processing.

- **Backend API Framework – FastAPI**

FastAPI is used to build high-performance backend APIs for handling document uploads, model communication, and output generation. It enables fast interaction between frontend and backend modules.

- **Database – MongoDB**

MongoDB is used to store uploaded documents, extracted invoice data, and generated outputs. It supports flexible storage of structured and semi-structured document information.

- **Frontend – React.js**

React.js is used to develop the user interface of the system. It allows users to upload invoices and receipts, view extracted results and download structured outputs through an interactive web application.

- **Image Processing – OpenCV**

OpenCV is used for preprocessing document images before OCR execution. It performs tasks such as resizing, noise removal, image enhancement, and contrast adjustment to improve text extraction accuracy.

- **Data Format – JSON**

JSON is used for storing and transferring extracted invoice information in a structured format. It enables smooth communication between APIs, frontend components, and database modules.

6.2 IMPLEMENTATION

6.2.1 User Interface Implementation

The frontend is developed using **React.js**. It allows the user to upload invoice or receipt files in image or PDF format. The interface also displays extracted data in a clear table or JSON format.

6.2.2 Document Upload Module

In this module, the user uploads the invoice, receipt, scanned bill, or photographed document. The uploaded file is sent to the backend through **FastAPI** for further processing.

6.2.3 Image Preprocessing Module

The uploaded document is preprocessed using **OpenCV**. It performs resizing, noise removal, brightness correction, and image enhancement to improve OCR accuracy.

6.2.4 OCR Text Extraction Module

Tesseract OCR / PaddleOCR is used to extract text from the uploaded document. The OCR engine converts image-based content into machine-readable text.

6.2.5 Layout Analysis Module

The **LayoutLMv3** is used to understand the structure of the document. It identifies important regions such as headers, tables, key-value pairs, and line items.

6.2.6 Field Extraction Module

The **Vision-Language Model (VLM)** extracts important invoice details such as vendor name, invoice number, date, tax amount, total amount, and item details.

6.2.7 Structured Output Generation

The extracted information is converted into **JSON format**. This makes the output organized and easy to display, store, and download.

6.2.8 Database Storage Module

MongoDB is used to store uploaded documents and extracted invoice data. This helps users access previous records whenever needed.

6.2.9 Result Display and Download

The final extracted result is shown in the frontend as a structured table or JSON output. The user can also download the result in formats such as PDF, JSON, or CSV.

CHAPTER 7

SCOPE FOR FUTURE ENHANCEMENTS

- **AI-Based Online Interview System**

The future version of the system can include an AI-powered online interview platform for students and job seekers. This module will allow users to attend mock interviews and improve their communication and technical skills. The AI system can simulate real company interview environments for better preparation.

- **Real-Time Mock and Technical Interviews**

The platform can conduct real-time technical and HR interviews directly within the application. Users can answer questions through text or voice interaction, and the system can evaluate their responses instantly. This feature helps users gain confidence and experience before attending actual interviews.

- **AI-Based Question Generation**

The system can automatically generate interview questions based on the user's skills, domain, and selected job role. For example, users applying for Python Developer or Data Analyst roles will receive role-specific technical questions. This makes interview preparation more personalized and effective.

- **Interview Performance Analysis**

The enhanced system can analyze user performance after every interview session. It can evaluate confidence, accuracy, response time, and technical knowledge to provide detailed feedback. This helps users understand their strengths and identify areas that need improvement.

- **Interview History and Improvement Tracking**

The platform can maintain a complete history of previous interview sessions, scores, and feedback reports. Users can track their improvement progress over time and compare performance between multiple sessions. This feature supports continuous learning and career development.

- **Personalized Career Guidance**

Future enhancements, the system can provide AI-based career suggestions and preparation tips. Based on interview performance and skill analysis, the platform can recommend courses, certifications, and important topics for improvement. This will help users prepare effectively for their target job roles..

- **Company-Specific Interview Preparation**

The platform can include company-specific interview preparation modules for companies like TCS, Infosys, Wipro, Google, and Amazon. Users can practice commonly asked questions and aptitude patterns based on the selected company. This enhancement increases the practical usefulness of the system for placement preparation.

CHAPTER 8

CONCLUSION

The proposed Multimodal Document Intelligence System provides an efficient and intelligent solution for processing invoices, receipts, forms, and other business documents using advanced AI technologies. The system combines OCR, Vision Language Models (VLMs), Layout-aware models, and Natural Language Processing techniques to accurately extract, understand, and organize document information.

By integrating text recognition with visual and layout understanding, the system overcomes the limitations of traditional document processing methods. The use of models such as PaddleOCR, LayoutLMv3, and multimodal AI techniques improves extraction accuracy even for complex document structures, scanned files, and low-quality images. The automated workflow reduces manual data entry, minimizes human errors, and increases processing speed..

The proposed system is highly beneficial for organizations, financial institutions, offices, and digital document management platforms where fast and accurate document processing is required. In addition, the system supports multiple document formats and enables intelligent information retrieval for better business operations..

REFERENCE

1. A. Kumar, R. Sharma, and P. Singh. (2026) “From Hallucination to Truth: A Survey on Fact-Checking in Large Language Models,” *IEEE Transactions on Artificial Intelligence*.
2. Z. Yang, H. Chen, and X. Liu. (2025) “JointCQ: Joint Claim and Query Generation for Fact Verification,” *arXiv preprint arXiv:2501.10234*.
3. H. Li, Y. Zhao, and M. Wang. (2024) “KnowHalu: Knowledge-based Hallucination Detection using External Sources,” *arXiv preprint arXiv:2403.11245*.
4. R. Sharma, A. Patel, and K. Verma. (2024) “Medico: Medical Hallucination Detection and Correction in Clinical Text,” *IEEE Access*, vol. 12, pp. 55421–55435.
5. T. Wang, S. Lee, and J. Kim. (2024) “DocLens: Claim-based Evaluation for Document-level Fact Verification,” in *Proceedings of ACL 2024*, pp. 2210–2221.
6. X. Manakul, A. Liusie, and M. Gales. (2023) “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative Models,” in *Proceedings of EMNLP 2023*, pp. 9004–9017.
7. L. Liu, Y. Xu, and P. Fung. (2023) “G-Eval: NLG Evaluation using GPT Models,” in *Proceedings of EMNLP 2023*, pp. 11120–11135.
8. Y. Chen, D. Li, and H. Zhao. (2023) “AlignScore: Evaluating Factual Consistency with Alignment Functions,” *arXiv preprint arXiv:2305.16739*.
9. J. Guo, F. Zhang, and C. Sun. (2022) “Improving Factual Consistency in Abstractive Summarization,” in *Proceedings of NAACL 2022*, pp. 1450–1462.
10. M. Lewis, Y. Liu, N. Goyal, et al. (2020) “BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation,” in *Proceedings of ACL 2020*, pp. 7871–7880.

11. T. Zhang, V. Kishore, F. Wu, et al. (2020) “BERTScore: Evaluating Text Generation with BERT,” in *Proceedings of ICLR 2020*.
12. K. Kryscinski, B. McCann, C. Xiong, and R. Socher. (2019) “FactCC: Fact Checking for Abstractive Summarization,” *arXiv preprint arXiv:1910.12840*.
13. I. Beltagy, K. Lo, and A. Cohan. (2019) “SciBERT: A Pretrained Language Model for Scientific Text,” in *Proceedings of EMNLP 2019*, pp. 3615–3620.
14. A. See, P. J. Liu, and C. D. Manning. (2017) “Get To The Point: Summarization with Pointer-Generator Networks,” in *Proceedings of ACL 2017*, pp. 1073–1083.
15. R. Nallapati, B. Zhou, C. Gulcehre, et al. (2016) “Abstractive Text Summarization using Sequence-to-Sequence RNNs and Beyond,” in *Proceedings of CoNLL 2016*, pp. 280–290.
16. M. Li, T. Lv, J. Cui, et al. (2021) “TrOCR: Transformer-based Optical Character Recognition,” in *Proceedings of ICCV 2021*.
17. S. Appalaraju, B. Jasani, B. Uyumazturk, and R. Manmatha. (2021) “DocFormer: End-to-End Transformer for Document Understanding,” in *Proceedings of ICCV 2021*.
18. N. Carion, F. Massa, G. Synnaeve, et al. (2020) “DETR: End-to-End Object Detection with Transformers,” in *Proceedings of ECCV 2020*.
19. A. Lee, H. Sung, and J. Kim. (2023) “Pix2Struct: Screenshot Parsing as Pretraining for Visual Language Understanding,” Google Research Technical Report.
20. J. Li, D. Li, S. Savarese, and S. Hoi. (2023) “BLIP-2: Bootstrapping Language-Image Pre-training with Frozen Image Encoders and Large Language Models,” in *Proceedings of ICML 2023*.

APPENDIX

A. CODING

Front-end (useSSEStream.js)

```
import 'package:flutter/material.dart';
import 'package:permission_handler/permission_handler.dart';
import 'screens/home_screen.dart';
const process = useCallback(async (files) => {
  // Start upload and get Job ID
  const { job_id } = await uploadFiles(files, token);

  // Connect to Server-Sent Events (SSE)
  const es = new EventSource(`/api/stream/${job_id}`);
  es.onmessage = (e) => {
    const msg = JSON.parse(e.data);
    if (msg.event === 'result') {
      setResults(prev => [...prev, msg.data]);
    } else if (msg.event === 'done') {
      setDone(true);
      es.close();
    }
  };
}, []);
```

Backend API (`backend/main.py`)

```
@app.post("/api/upload")
async def upload(files: List[UploadFile] = File(...), user_email: str =
Depends(get_current_user_optional)):
    job_id = str(uuid.uuid4())
```



```

    target_dir = (UPLOAD_DIR / user_email) if user_email else
(UPLOAD_DIR / "tmp" / job_id)
    target_dir.mkdir(parents=True, exist_ok=True)
    # ... logic to save files ...
    return {"job_id": job_id}

@app.get("/api/stream/{job_id}")
async def stream(job_id: str):
    async def gen():
        for info in job["files"]:
            # Run the full extraction pipeline for each file
            res = await run_pipeline(info["path"], info["bytes"],
info["name"], user_email=job["user_email"], session_id=job_id)
            yield f"data: {json.dumps({'event': 'result', 'data': res},
ensure_ascii=False)}\n\n"
            yield f"data: {json.dumps({'event': 'done'})}\n\n"
        return StreamingResponse(gen(), media_type="text/event-stream")

```

Image Preprocessing(image_enhancer.py)

```

def split_for_extraction(image_bytes: bytes) -> list[bytes]:
    """Divide & Conquer: Splits large/tall images into meaningful
segments."""
    img = Image.open(io.BytesIO(image_bytes)).convert("RGB")
    w, h = img.size
    if w > h * 1.8: # Wide Side-by-Side
        left = img.crop((0, 0, w // 2, h))
        right = img.crop((w // 2, 0, w, h))
        return [enhance_for_vlm_pil(left), enhance_for_vlm_pil(right)]
    elif h > w * 1.5: # Tall Overlapping Split

```

```

top = img.crop((0, 0, w, int(h * 0.55)))
bottom = img.crop((0, int(h * 0.45), w, h))
return [enhance_for_vlm_pil(top), enhance_for_vlm_pil(bottom)]
return [enhance_for_vlm(image_bytes)]

```

```

def _invert_colored_bands(img: Image.Image) -> Image.Image:
    """Inverts dark colored headers (white text on dark bg) to improve
    OCR accuracy."""
    # ... HSV analysis logic ...
    # Only inverts colored regions, leaves white background alone.
    return Image.fromarray(result)

```

VLM Model Interaction (vlm_model.py)

```

# Language-specific context rules for multi-script support
_LANG_SPECIFIC_RULES = {
    "hindi": "The text uses Devanagari script. Read matras (vowel
marks) carefully.",
    "tamil": "The text uses Tamil script. Letters have circular curves.",
    "telugu": "The text uses Telugu script. Characters have rounded
shapes.",
    "urdu": "The text uses Urdu/Nastaliq script (right-to-left).",
}

```

```

def _clean_output(text: str) -> dict | None:
    # Strip markdown fences and find the JSON block
    text = text.replace("```json", "").replace("```", "").strip()
    start, end = text.find('{'), text.rfind('}')
    if start != -1 and end != -1:
        text = text[start:end+1]

```

```
return {"fields": json.loads(text)}
```

Layout Template Mapping (layout_template.py)

```
INVOICE_TEMPLATE = [  
    ("vendor_name", "top_left", ["vendor", "shop", "seller",  
    "company"]),  
    ("invoice_date", "top_right", ["date", "bill_date", "dated"]),  
    ("items", "center", ["items", "particulars", "description"]),  
    ("total_amount", "bottom_right", ["total", "grand_total", "payable"]),  
]
```

```
def map_to_standard_template(raw_extraction: dict):  
    # Uses fuzzy alias matching to map raw fields to the canonical  
    template  
    result = {}  
    for field_name, zone, aliases in INVOICE_TEMPLATE:  
        val = _find_value(raw_extraction, aliases)  
        result[field_name] = val or ""  
    return result
```

Remote Kaggle Routing (gguf_engine.py)

```
def query_local_llava(image_bytes, prompt, ...):  
    kaggle_url = _get_dynamic_kaggle_url()  
    if kaggle_url:  
        # Route to Kaggle API  
        b64 = base64.b64encode(image_bytes).decode("utf-8")  
        job_id = start_remote_job(kaggle_url, b64, prompt)  
        # Poll for completion  
        while True:
```

```

res = check_status(kaggle_url, job_id)
if res["status"] == "completed": return res["data"]
time.sleep(5)

```

```

# Local Inference fallback
client = _load_gguf_model()
return client.create_chat_completion(...)

```

Database Persistence (db/repository.py)

```

async def save_result(result: dict[str, Any]) -> str | None:

```

```

    """Persist a pipeline result dict to MongoDB."""

```

```

    db = get_async_db()

```

```

    if db is None: return None

```

```

    doc = make_invoice_document(
        file_name=result.get("image_name"),
        vlm_fields=result.get("vlm_fields"),
        vlm_source=result.get("vlm_source"),
        status="success"
    )

```

```

    try:

```

```

        inserted = await db["invoice_results"].insert_one(doc)

```

```

        return str(inserted.inserted_id)

```

```

    except Exception as exc:

```

```

        return None

```

B.SCREENSHOTS:

HOME SCREEN:

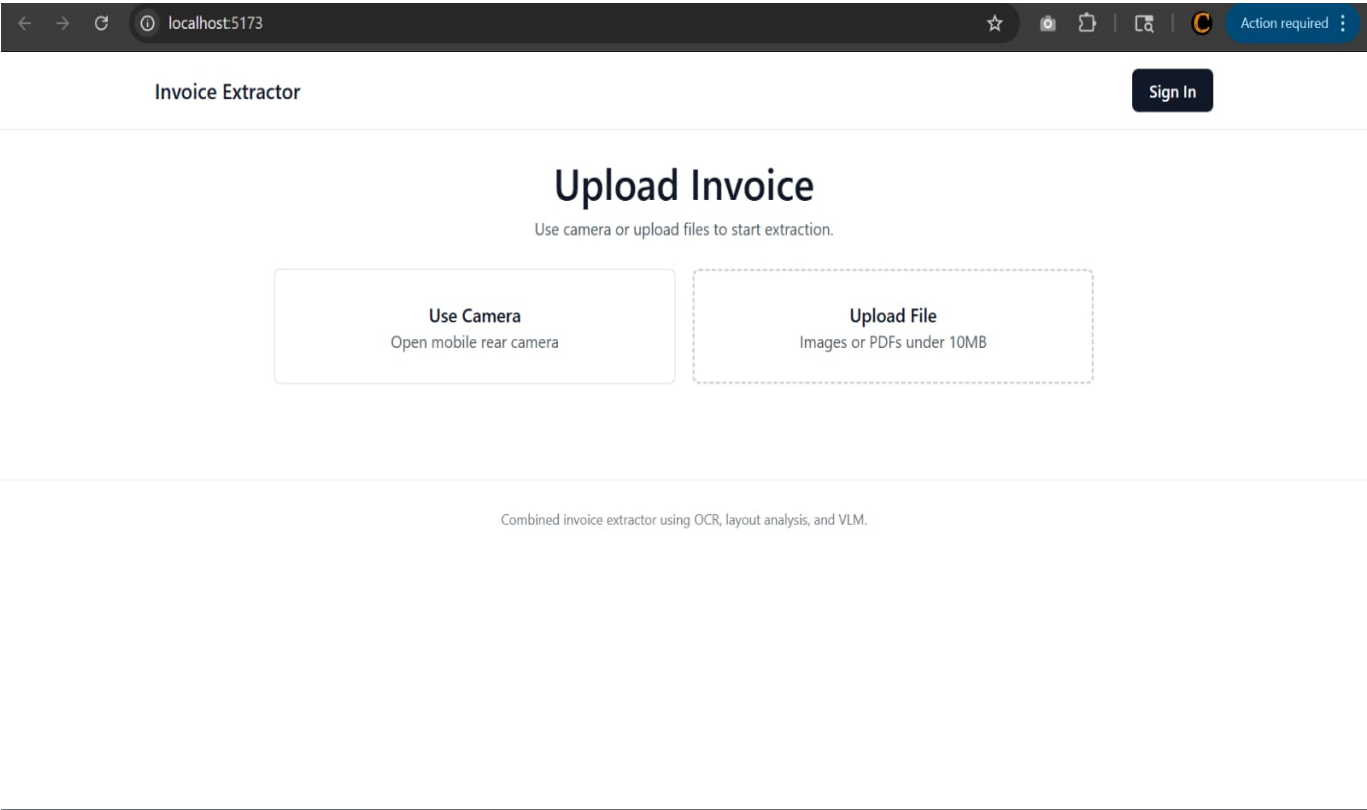


Fig.No:B.1 Home Screen

The homepage provides a simple and user-friendly interface for uploading invoice images or PDF files through camera capture or file selection. The system processes the uploaded documents using OCR, layout analysis, and Vision Language Models to extract important invoice information accurately and efficiently.

INVOICE UPLOAD SCREEN:

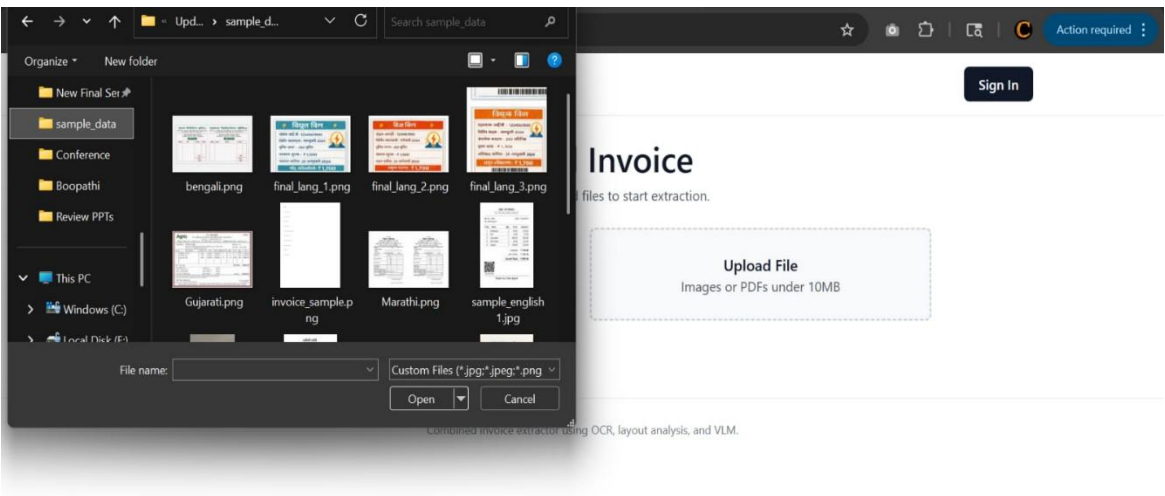


Fig.No:B.2 Invoice Upload Screen

This image shows the file upload interface of the AI-based Invoice Extractor system. Users can select invoice images or PDF files from their local device to upload and start the OCR, layout analysis, and VLM-based extraction process.

PROCESSING SCREEN:

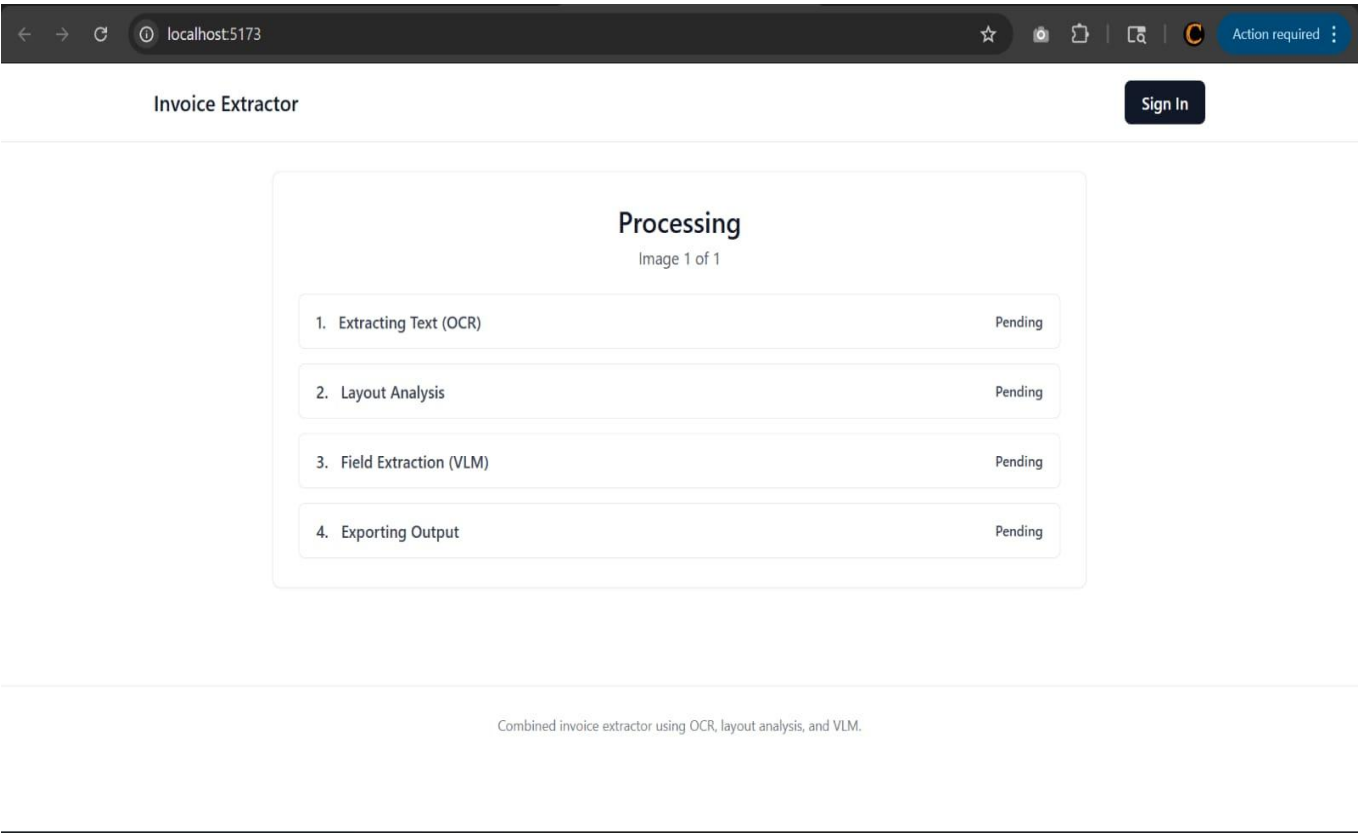


Fig.No:B.3 Processing Screen

This image shows the processing workflow of the AI-based Invoice Extractor system. The system performs OCR text extraction, layout analysis, field extraction using VLM, and finally exports the structured invoice output.

OUTPUT SCREEN:

The screenshot displays a web browser window at localhost:5173. The application is titled "Invoice Extractor" and includes a "Sign In" button. It features two side-by-side panels for output. The left panel, "NATIVE SCRIPT OUTPUT", shows a cash memo in Marathi with details like "Rajwar Onward Computer Centre" and a list of services including Internet, Game, Print, and Scan. The right panel, "ENGLISH TRANSLATION OUTPUT", provides the same information in English. Both panels include fields for Bill No., Date, Customer Name, Address, and Login/Logout times. A table lists services with columns for Particulars, Quantity, Rate (Rs.), and Amount (Rs.). The bottom of the interface contains a note about the distributed architecture.

Invoice Extractor Sign In

NATIVE SCRIPT OUTPUT

Cash Memo
मैजिक कॉम्प्युटर
राजवर ऑफिस कॉम्प्युटर एन्टरप्राइज
केयर मनेजमेंट, विक्रम म. 701, आला म. 20
(मार्गिला माहाला), बिन माधव डार, फि. विपुर्द, 416520.
Phone: (02362) 221116, Mob.: 8975065407.

Bill No.: Date: / /20
Customer Name:
Address:
Login Time: am/pm Logout Time: am/pm

Particulars	Quantity	Rate (Rs.)	Amount (Rs.)
Internet Service	hr min		
Game Service	hr min		
Print Pages			
Scan Pages			
Marathi / English Typing			
Other Additional Charges (like Inverter, etc.)			
Total Amount			
Total Amount In Words:			

For, Magic Computer
Proprietor / Authorized Sign

ENGLISH TRANSLATION OUTPUT

Cash Memo
Magic Computer
Rajwar Onward Computer Centre
Care Management, Vikram M, 701, Aala M. 20
(Margila Mahal), Bin Madhav Dar, Fi. Vipur, 416520.
Phone: (02362) 221116, Mob.: 8975065407.

Bill No.: Date: / /20
Customer Name:
Address:
Login Time: am/pm Logout Time: am/pm

Particulars	Quantity	Rate (Rs.)	Amount (Rs.)
Internet Service	hr min		
Game Service	hr min		
Print Pages			
Scan Pages			
Marathi / English Typing			
Other Additional Charges (like Inverter, etc.)			
Total Amount			
Total Amount In Words:			

For, Magic Computer
Proprietor / Authorized Sign

Using Distributed Architecture: Lightweight layout passes run locally, while the heavy 32B semantic extraction runs securely on Kaggle Cloud.

Fig.No:B.4 Output Screen

This image shows an AI-based Invoice Extractor system that extracts invoice text from a native language document using OCR technology. It also translates the extracted content into English and displays both outputs side by side for easy understanding